

TECHNICAL REFERENCE GUIDE

Qwik Guide: Supervised Learning

Classification · Regression · Trees · Forests · Logistic Regression · SVM · Naïve Bayes

A beginner-friendly companion for ML for Beginners

Supervised Learning, From the Ground Up

Supervised learning is one of the most widely used techniques in machine learning. You give an algorithm a set of **labeled examples** — inputs paired with the correct answers — and it learns a rule that maps new inputs to answers. This guide walks a beginner through the core supervised models in a sensible learning order, keeps the language plain, and adds diagrams wherever a picture helps. Each section ends with the key idea you should remember.

TIP You don't have to master every model at once. Start with the difference between **classification** and **regression** (Section 1), then work down the guide. Each model builds on vocabulary introduced earlier.

1 Classification vs. Regression

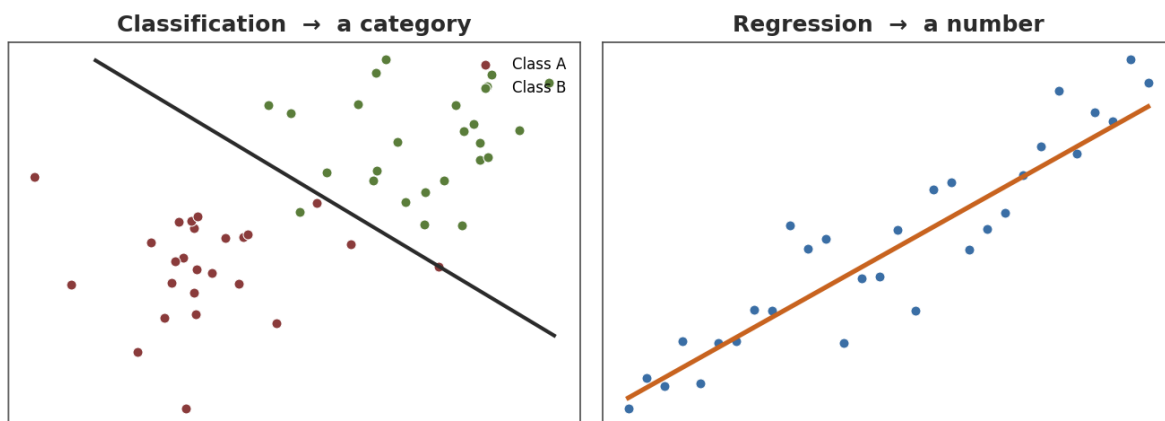
Every supervised problem is either a classification problem or a regression problem. Telling them apart is the first skill to build, because it decides which models you can use.

Classification — predicting a category

The model maps each input to one of a finite set of classes. **Binary classification** has two classes (yes/no, spam/not-spam). **Multiclass classification** has more than two — for example, sorting movies into drama, comedy, horror, or romance.

Regression — predicting a number

The model maps each input to a numerical value instead of a class — predicting a house price from its area, room count, and lot size, for example. A model can also estimate a *probability* (say, an 80% chance an email is spam), which you can then threshold into a class label; regression is the broader category that covers any numeric output.



Classification draws boundaries between classes; regression fits a line (or curve) through the values.

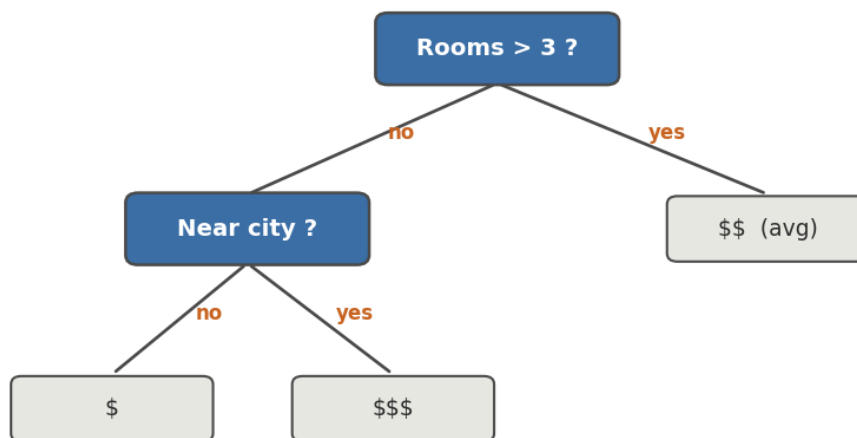
You can sometimes reformulate one as the other. A model that outputs a probability becomes a classifier once you pick the highest-probability class — in handwriting recognition, an 80% score for the letter “a” becomes the prediction “a”. Going the other way, you can turn numbers into classes with ranges: incomes of 0–20,000, 20,000–60,000, and so on.

TIP Beyond these two there are **time-series** problems, where examples are ordered in time and each value depends on past ones — weather, stock prices, or predicting the next video frame. These use **sequence models**, of which hidden Markov models and recurrent neural networks are two common examples among many.

2 Decision Trees & Regression Trees

A decision tree splits the data with a series of simple yes/no questions until each group is pure enough to make a prediction. The same idea works for both classification and regression.

A decision tree splits the data into regions, then predicts



Each question partitions the data; each leaf holds a prediction. A regression tree returns a number at the leaf, a classification tree returns a class.

Regression tree

Predicts a numerical value. The simplest kind — a **piecewise-constant** tree — returns a single constant at each leaf: the average of the training values that landed in that partition. If a value is missing, it returns the average of the branches that value could lead to.

Trees vs. linear regression. A regression tree applies a model *locally*, inside small partitions, and can capture interactions easily — the value of an extra bedroom might matter more in one city than another. Linear regression instead applies one model *globally* across all the data and can't capture those interactions as naturally.

How a tree learns (sum of squared errors)

For regression, the tree measures error with the sum of squared errors, $S = \sum (y_j - \mu_j)^2$. It computes S for the current data, chooses the split that reduces S the most, splits, and repeats — stopping when no split helps or nothing is left to divide.

3 CART vs. C4.5 — How Splits Are Chosen

Decision trees are grown top-down. At each step the algorithm picks the variable that best separates the data, using a scoring metric. Two classic algorithms make that choice differently.

CART (Classification and Regression Trees)

Makes **binary** splits only (two branches at a time) and scores them with **Gini impurity**, $\text{Gini} = 1 - \sum p_i^2$ (equivalently $\sum p_i(1 - p_i)$) — the probability of misclassifying a randomly chosen item. It is CART's split criterion, distinct from information gain.

C4.5

Allows **non-binary** splits and scores them with **information gain** — the **entropy before the split** minus the **weighted entropy of the branches after the split**: $\text{Gain} = H(S) - \sum (|T| / |S|) \cdot H(T)$. Entropy measures uncertainty, so the bigger the drop, the better the split.

TIP Quick memory hook: **C4.5** → **entropy / information gain**; **CART** → **Gini impurity**; **regression trees** → **sum of squared errors**. Same tree-building loop, different rulers for measuring a split.

Pruning

After a tree is grown it is often **pruned** — leaves and branches are removed as long as accuracy doesn't drop noticeably. Pruning cuts computational cost and, done selectively, reduces **overfitting** (the tree memorizing noise instead of learning the pattern).

4 Random Forests, Bagging & Bootstrapping

A single deep tree overfits easily. Random forests fix this by training many trees on slightly different slices of the data and letting them vote. Three terms make this work.

Bootstrapping

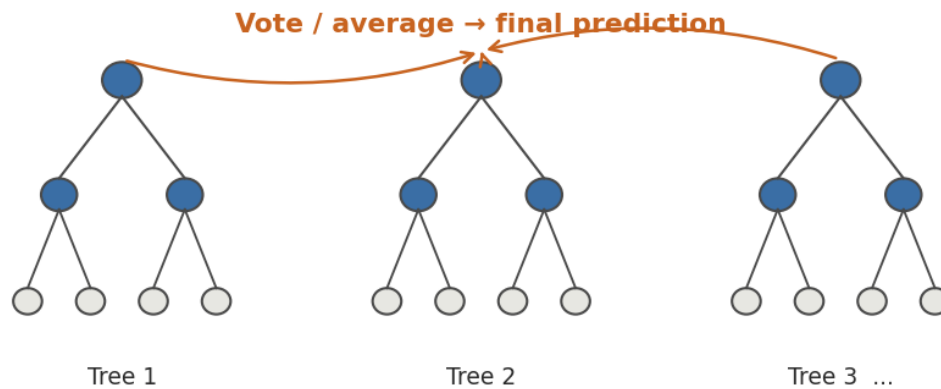
Sampling **with replacement** from the original dataset to build several subsamples of the same size. Because draws are with replacement, the same example can appear more than once in a subsample and subsamples overlap — each tree sees a slightly different view of the data.

Bagging (bootstrap aggregation)

Fit a separate model to each subsample, then combine them: average the predictions for regression, or take a majority vote for classification — a democratic process across many models.

Feature bagging

The key extra ingredient in a random forest: at **each split**, the tree may only choose from a **random subset of the features**, not all of them. This stops every tree from leaning on the same strong predictor. Without it, if one feature is very predictive, most trees put it at the root and the trees end up too similar to add value when combined.



A random forest: many trees, each grown on a random subsample and a random subset of features, then combined by voting or averaging.

The random forest recipe:

- 1 Take n bootstrap subsamples of the dataset (sampling with replacement).
- 2 Grow a **CART-style** tree on each subsample.
- 3 At **every split**, consider only a random subset of features (feature bagging) — this is the key distinction from plain bagging, and it decorrelates the trees.
- 4 To predict, average the trees' outputs (regression) or take a majority vote (classification).

```
from sklearn.ensemble import RandomForestClassifier

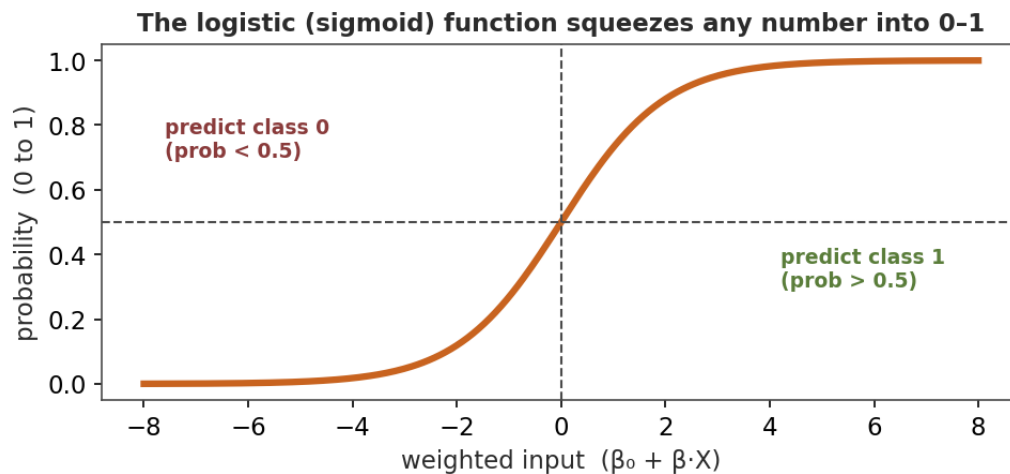
model = RandomForestClassifier(n_estimators=100, random_state=0)
model.fit(X, y)
preds = model.predict(X_new)
```

5 Logistic Regression

Despite the name, logistic regression is a **classification** model. It is widely used for binary classification because the algorithm is simple and always returns a number between 0 and 1 — which we read as a probability.

The model

Logistic regression estimates the **probability** that an input belongs to a class. It runs a linear combination of the inputs through the **logistic (sigmoid) function**, $f(x) = 1 / (1 + e^{-x})$, which squeezes any value into the range 0 to 1. A **threshold** (commonly 0.5) then turns that probability into a class label — so the final output is a probability first, and a yes/no decision second.



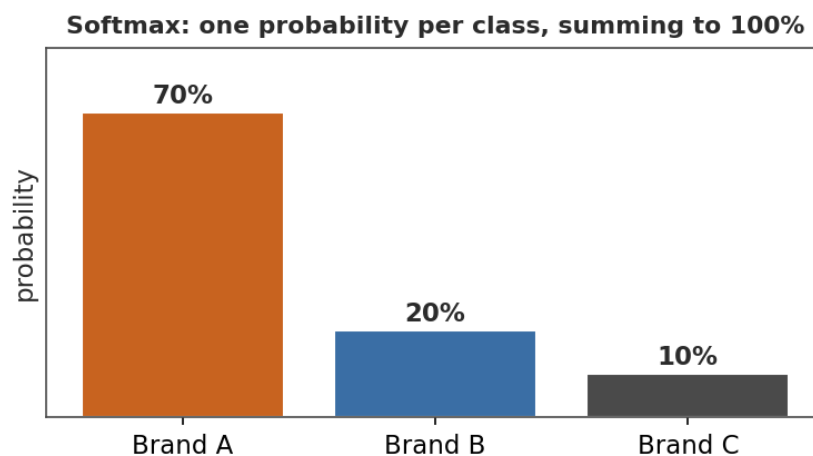
The sigmoid curve. Inputs above the 0.5 threshold are predicted as one class, below it as the other.

Working with multiple classes

With more than two classes, you can **compare classes against each other** — e.g. with three brands, run A-vs-B, A-vs-C, B-vs-C and keep the one that wins most — or **group** one class versus all the rest (“one-vs-rest”).

Softmax regression

A **separate multiclass extension** of logistic regression — not the standard binary output. Where binary logistic regression returns one probability, softmax returns a probability for every class, all summing to 100%, and you pick the highest.



Softmax output: a probability for every class, adding up to 100%.

TIP Models are trained with **gradient descent**, an iterative optimization method that nudges the weights step by step to reduce error. The update rule for logistic regression looks much like the one for linear regression.

6 Support Vector Machines (SVM)

An SVM separates two classes by drawing the best possible dividing boundary between them, then uses that boundary to classify new points.

Hyperplane

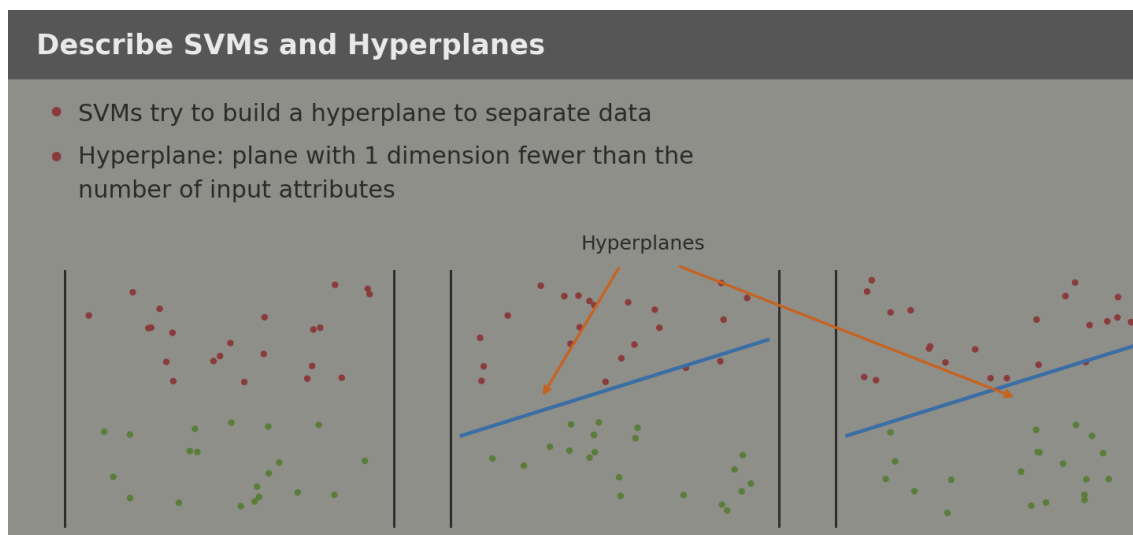
The dividing boundary. A hyperplane is a plane with **one dimension fewer** than the number of input attributes — in a 2-D space (two attributes) the hyperplane is simply a line.

Optimal hyperplane and the margin

The best hyperplane has the largest **margin** — the widest gap between itself and the nearest points of either class. A wide margin generalizes better to new data.

Support vectors

The **input vectors closest to the hyperplane**. They sit right at the edge of the margin and are the only points that define where the boundary goes — move them and the boundary moves.



SVMs build a hyperplane to separate data; the best one leaves the widest margin. (Illustration for ML for Beginners.)

Soft-margin hyperplane

Real data is rarely perfectly separable. A soft-margin SVM uses a **cost** instead of a hard condition on the margin, allowing a few points to sit on the wrong side. It relies on a **hinge loss** function that adds up how far the misplaced points are from the margin; training picks the SVM with the smallest hinge loss.

Kernel methods

When classes can't be separated by a straight boundary, a **kernel** adds dimensions (features) so they can be. Common kernels: **linear**, **polynomial** (e.g. squaring inputs), and the **radial basis function (RBF)**, a popular Gaussian kernel.

```
from sklearn import svm

clf = svm.SVC()
clf.fit(X, y)           # X = samples x features, y = class labels
clf.predict([[2., 2.]]) # classify a new point
clf.support_vectors_    # the points that define the boundary
```

7 Naïve Bayes Classifiers

Naïve Bayes is a fast, surprisingly effective classifier built on a single idea from probability theory — and one bold simplifying assumption.

Bayes' theorem

A method that defines the **probability of events using previously established understanding** of related factors. If thunder and lightning are linked to rain, knowing whether it will rain sharpens your estimate of thunder. Formally: $P(A|B) = P(B|A) \cdot P(A) / P(B)$.

The “naïve” assumption

The classifier assumes **all features are independent of each other**. With **bag-of-words** text inputs — where a document is represented purely by its word counts — this means word order is ignored, so “David buys a potato” is treated the same as “potato buys a David”. The assumption is rarely true, yet the model works well in practice.

To classify a document, Naïve Bayes computes the probability of each class given the input and picks the highest — a *maximum a posteriori* estimate, i.e. the class with the most evidence. Because multiplying many tiny probabilities causes floating-point errors, implementations sum **log-probabilities** instead, using $\log(a \cdot b) = \log(a) + \log(b)$. A small “add-one” count is added to every term so that a word never seen in a class doesn't force a probability of zero.

```
from sklearn.naive_bayes import MultinomialNB

clf = MultinomialNB()
clf.fit(X, y)
clf.predict(X_new)
```

TIP Naïve Bayes shines for text — spam filtering, topic labeling, sentiment — where inputs are word counts and there are many features. It's a strong, cheap baseline to try before heavier models.

8 Quick Reference

Model	Problem type	Remember this
Regression tree	Regression	Local models in partitions; leaf returns an average

CART	Both	Binary splits, scored by Gini impurity
C4.5	Both	Non-binary splits, scored by entropy / info gain
Random forest	Both	Many trees + bagging + feature bagging, then vote
Logistic regression	Classification	Sigmoid → probability, then threshold to a label (softmax is a separate multiclass extension)
SVM	Classification	Widest-margin hyperplane; support vectors define it
Naïve Bayes	Classification	Bayes' theorem + assume all features independent

★ Further Resources

The links below are arranged in a recommended learning order — foundations first, then regression, classification, data preparation, evaluation, and finally hands-on projects.

Foundations

Microsoft: Introduction to ML Concepts —

<https://learn.microsoft.com/en-us/training/modules/fundamentals-machine-learning/>

Google Machine Learning Crash Course — <https://developers.google.com/machine-learning/crash-course>

Google: Prerequisites and Prework —

<https://developers.google.com/machine-learning/crash-course/prereqs-and-prework>

Scikit-learn MOOC (free, beginner-friendly) — <https://inria.github.io/scikit-learn-mooc/>

Regression — Predicting Numbers

Google: Linear Regression —

<https://developers.google.com/machine-learning/crash-course/linear-regression>

Microsoft: Introduction to Regression — <https://learn.microsoft.com/en-us/shows/machine-learning-for-beginners/introduction-to-regression-models-for-machine-learning-machine-learning-for-beginners>

Microsoft ML for Beginners: Regression —

<https://github.com/microsoft/ML-For-Beginners/tree/main/2-Regression>

Scikit-learn: Linear Models — https://scikit-learn.org/stable/modules/linear_model.html

Classification — Predicting Categories

Google: Logistic Regression —

<https://developers.google.com/machine-learning/crash-course/logistic-regression>

Google: Classification (thresholds, precision, recall, ROC, AUC) —

<https://developers.google.com/machine-learning/crash-course/classification>

Microsoft ML for Beginners: Classification —

<https://github.com/microsoft/ML-For-Beginners/tree/main/4-Classification>

Scikit-learn: Classification Examples —

https://scikit-learn.org/stable/auto_examples/classification/index.html

Preparing Data

Google: Working with Numerical Data —

<https://developers.google.com/machine-learning/crash-course/numerical-data>

Google: Working with Categorical Data —

<https://developers.google.com/machine-learning/crash-course/categorical-data>

Scikit-learn: Preprocessing Data — <https://scikit-learn.org/stable/modules/preprocessing.html>

Scikit-learn: Common Pitfalls (data leakage) — https://scikit-learn.org/stable/common_pitfalls.html

Training & Evaluating Models

Google: Datasets, Generalization & Overfitting —

<https://developers.google.com/machine-learning/crash-course/overfitting>

Scikit-learn: Model Selection & Evaluation — https://scikit-learn.org/stable/model_selection.html

Scikit-learn: Getting Started — https://scikit-learn.org/stable/getting_started.html

Google ML Exercises — <https://developers.google.com/machine-learning/crash-course/exercises>

Key beginner metrics. Regression: MAE, MSE, RMSE, R^2 . Classification: accuracy, precision, recall, F1 score, confusion matrix.

Algorithms & Practice

Scikit-learn: Supervised Learning Guide — https://scikit-learn.org/stable/supervised_learning.html

UCI Iris Dataset (great first project) — <https://archive.ics.uci.edu/dataset/53/iris>

UCI Dataset Collection — <https://archive.ics.uci.edu/datasets>

Scikit-learn Example Gallery — https://scikit-learn.org/stable/auto_examples/index.html

Microsoft ML for Beginners (12-week curriculum) — <https://github.com/microsoft/ML-For-Beginners>

TIP Recommended path. For one structured starting point, use the free **Scikit-learn MOOC**. Supplement hard concepts with the **Google ML Crash Course**, then build projects from **Microsoft ML for Beginners** — try predicting Iris species, house prices, or spam, and compare logistic regression, a decision tree, and a random forest on one dataset.